

Performance Testing

Date	18 July 2026
Team ID	SWTID-2026-5023
Project Name	Rising Waters (Flood Prediction System)
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	JMeter / Postman
Type of Testing	Load Testing
Target Module / API	Flood Risk Prediction API
Test Environment	Local System (Python + Flask)
Test Date	18 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Predict flood risk for single input	10	60 sec	Response within 2 sec
2	Multiple prediction requests	25	60 sec	Stable performance
3	Large dataset prediction	50	120 sec	No Crashes
4	Invalid input handling	10	30 sec	Error message displayed

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 sec	Pass	Good
2	Response Time (Max)	< 5 seconds	3.1 sec	Pass	Acceptable
3	Throughput (Req/sec)	20 Req/sec	22 Req/sec	Pass	Acceptable
4	Error Rate	< 1%	0%	Pass	No errors
5	CPU Utilization	< 80%	58%	Pass	Normal
6	Memory Utilization	< 80%	64%	Pass	Stable

Step 4: Observations & Analysis

Key Findings:

- The flood prediction system responded quickly under normal load.
- The model generated accurate predictions without errors.
- Response time remained within acceptable limits.

Bottlenecks Identified:

- Slight delay when processing multiple requests simultaneously.

Optimization Steps Taken:

- Optimized data preprocessing.
- Improved model loading.
- Reduced unnecessary computations.

Step 5: Screenshots / Evidence

The screenshot shows a JupyterLab environment with a file explorer on the left and a terminal window in the center. The file explorer displays a project structure for 'flood_model.py' with various files and folders. The terminal window shows the output of running 'python train_model.py', which includes the dataset shape, a list of missing values for various features, and a summary of the XGB model performance. The bottom status bar indicates 'Best model (XGB) trained and saved successfully!'.

File Explorer Structure:

- EXPLORER
 - train_model.py 5 X
 - train_model.py 2
 - dataset
 - flood.csv
 - graphs
 - correlation_heatmap
 - scatter_plot.png
 - model
 - flood_model.pkl
 - static
 - css
 - style.css
 - images
 - templates
 - index.html
 - result.html
 - app.py
 - model_comparison.py
 - README.md
 - requirements.txt
 - train_model.py 5
 - visualization.py

Terminal Output:

```
PS C:\Users\Uthavna\Downloads\Flooding_Waters> python train_model.py
dataset Shape:
(50000, 21)

Missing Values:
PavementIntensity      0
TopographyDrainage     0
RiverManagement       0
Deforestation          0
Urbanization           0
ClimateChange         0
DamQuality             0
Siltation              0
AgriculturalPractices  0
Encroachments          0
IneffectiveDisasterPreparedness  0
DrainageSystems        0
CoastalVulnerability   0
Landslides             0
Watersheds             0
DeterioratingInfrastructure  0
PopulationCore         0
Wetlands               0
InadequatePlanning     0
PoliticalFactors        0
FloodProbability       0
dtype: int64

----- XGB MODEL -----
RMSE : 0.0186357
R2 Score : 0.7772116654528751

Best model (XGB) trained and saved successfully!
```

← 127.0.0.1:5000/predict Chat

 **Rising Waters**
Flood Prediction Result

Flood Probability

0.337

Flood Risk Level

 **Moderate Flood Risk**

The predicted flood probability is moderate. Stay alert and monitor weather updates.

 Predict Again